

INDEX

Sr No.	Title	Page no
1	Introduction to Node.js, Modules and Events	2
2	File Handling& HTTP Web Server	11
3	Databases	18
4	Introduction to ReactJs	25
5	Components and Events	27

Practical 1: Introduction to Node.js, Modules and Events

1. What is Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment that allows JavaScript to be executed outside the browser. It is built on Google Chrome's V8 JavaScript engine and is mainly used for building fast, scalable network applications.

2. Advantages of Node.js

1. Uses non-blocking, asynchronous I/O
2. High performance due to V8 engine
3. Single-threaded but highly scalable
4. Fast execution of code
5. Large ecosystem of libraries via NPM
6. Suitable for real-time applications
7. Supports event-driven programming
8. Easy to learn for JavaScript developers

3. Node.js Process Model

Node.js follows a single-threaded, event-driven, non-blocking model.

Working:

- Client sends request
- Request goes to Event Queue
- Event Loop processes requests
- Non-blocking operations are handled asynchronously
- Response is sent back to client

This model improves performance and scalability.

4. Traditional Web Server Model

Traditional servers (like Apache) use a multi-threaded blocking model.

Limitations:

- Each request creates a new thread
- High memory usage
- Slower performance under heavy load
- Not suitable for real-time apps

5. Node.js vs Traditional Web Server Model

Traditional Model	Node.js Model
Blocking I/O	Non-blocking I/O
Multi-threaded	Single-threaded
High memory usage	Low memory usage
Slow for real-time apps	Fast & scalable

6. Setup Development Environment

Installation of Node.js on Windows

1. Visit official Node.js website
2. Download LTS version
3. Run the installer
4. Follow setup wizard
5. Verify installation:

node -v

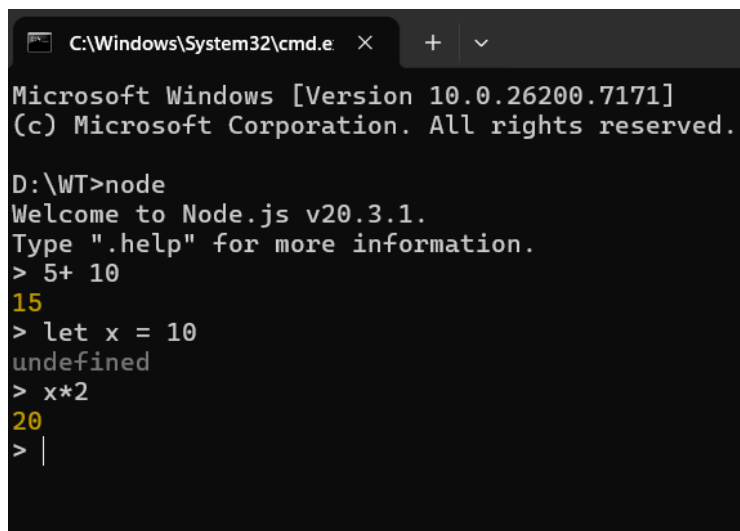
npm -v

7. Working in REPL

REPL stands for Read-Eval-Print-Loop.

Steps to start REPL:

node

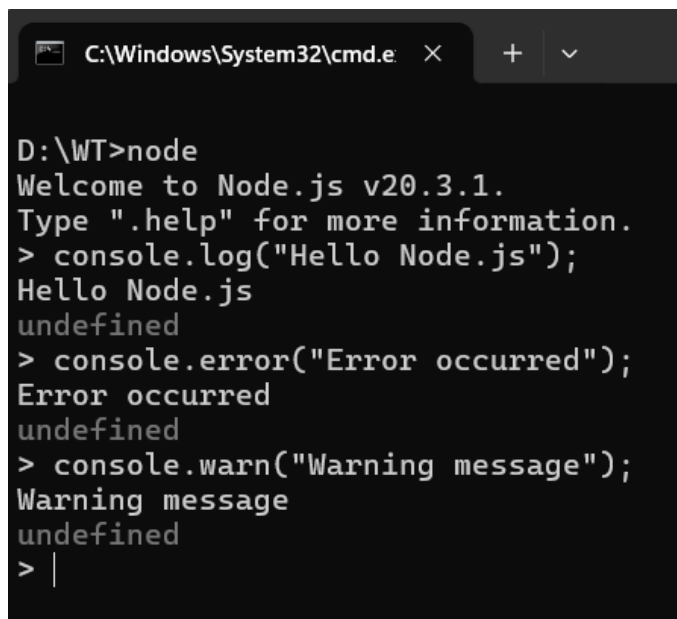
A screenshot of a Windows Command Prompt window. The title bar shows 'C:\Windows\System32\cmd.e' with a close button and a dropdown arrow. The window content shows the following text:

```
Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.

D:\WT>node
Welcome to Node.js v20.3.1.
Type ".help" for more information.
> 5+ 10
15
> let x = 10
undefined
> x*2
20
> |
```

8. Node.js Console

The console module is used for output and debugging.

A screenshot of a Windows Command Prompt window. The title bar shows the path 'C:\Windows\System32\cmd.e' and standard window controls. The command prompt shows the following sequence of commands and outputs:

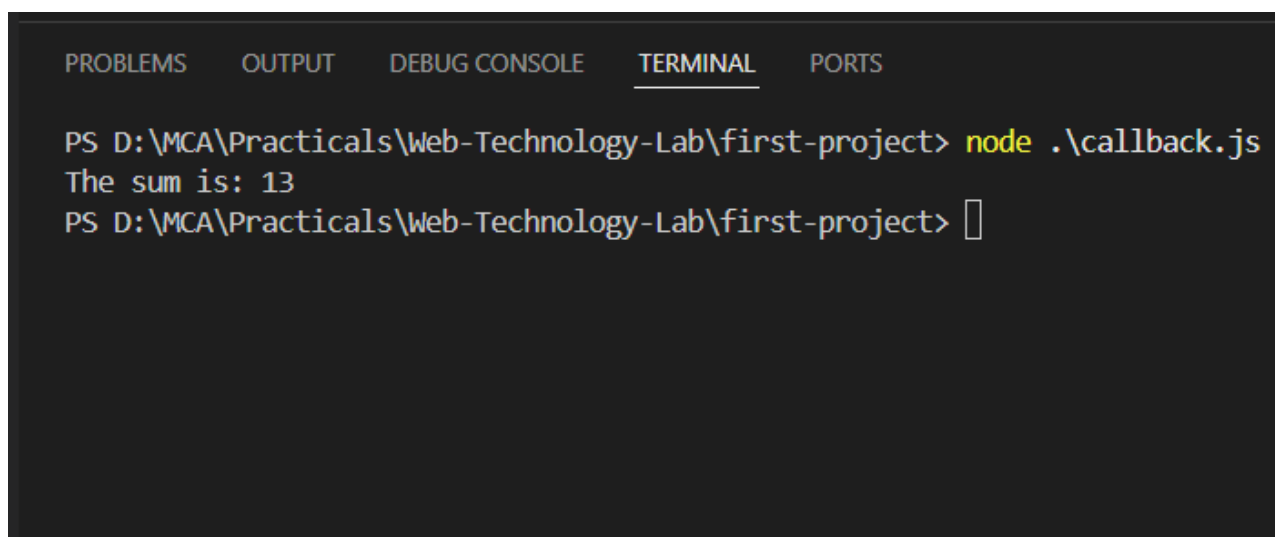
```
D:\WT>node
Welcome to Node.js v20.3.1.
Type ".help" for more information.
> console.log("Hello Node.js");
Hello Node.js
undefined
> console.error("Error occurred");
Error occurred
undefined
> console.warn("Warning message");
Warning message
undefined
> |
```

9. Standard Callback Pattern

```
function addNO(a, b, callback) {
    let sum = a + b;
    callback(sum);
}
```

```
function display(result) {
    console.log("The sum is: " + result);
}
```

```
addNO(5, 8, display);
```

A screenshot of a VS Code terminal window. The top bar shows tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. The terminal shows the following commands and output:

```
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> node .\callback.js
The sum is: 13
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> |
```

10. Event Emitter Pattern

```
const EventEmitter = require("events");

const emitter = new EventEmitter();

emitter.on("Employ", (username, Age, Dept, JD) => {
  console.log("Employ", username, Age, Dept, JD);
});

emitter.emit(
  "Employ",
  "XYZ",
  "is 25yrs old",
  "Appoint in I.T Dept",
  "and joined on Feb 2024"
);
```

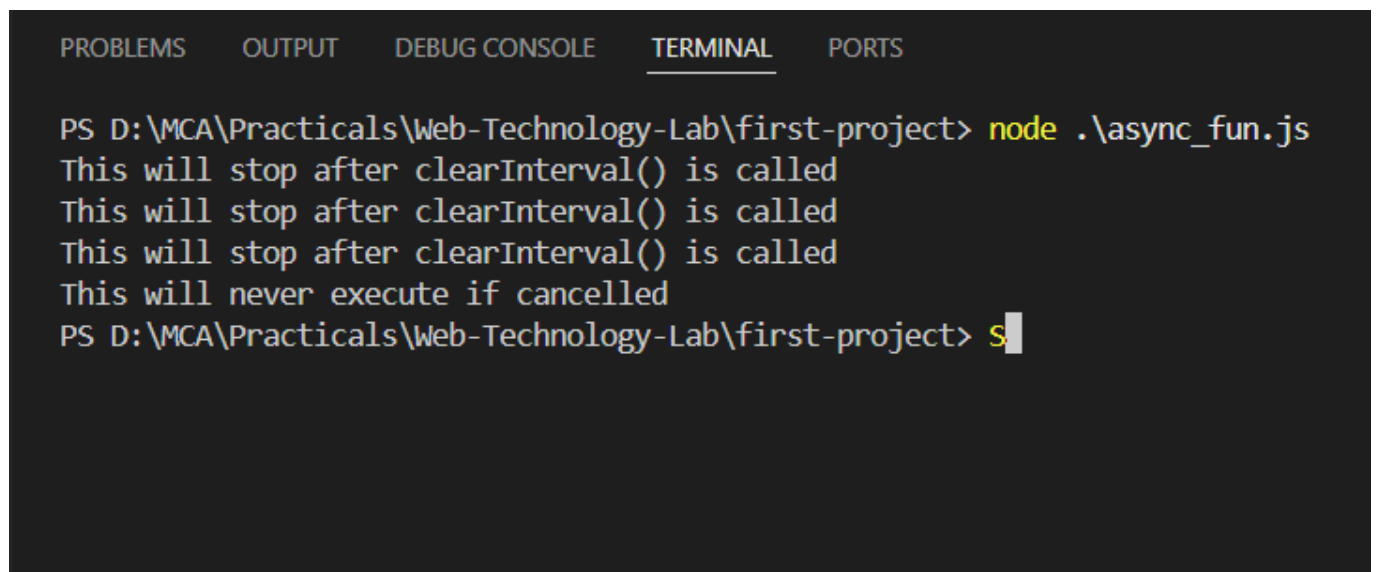


The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The terminal content shows a PowerShell prompt 'PS D:\MCA\Practicals\Web-Technology-Lab\first-project>' followed by the command 'node .\emit.js'. The output of the command is 'Employ XYZ is 25yrs old Appoint in I.T Dept and joined on Feb 2024'. A second PowerShell prompt 'PS D:\MCA\Practicals\Web-Technology-Lab\first-project>' is shown at the bottom with a cursor.

11. Defer Execution and Cancel Execution of a Function

```
let intervalId = setInterval(() => {  
    console.log("This will stop after clearInterval() is called");  
}, 1500);
```

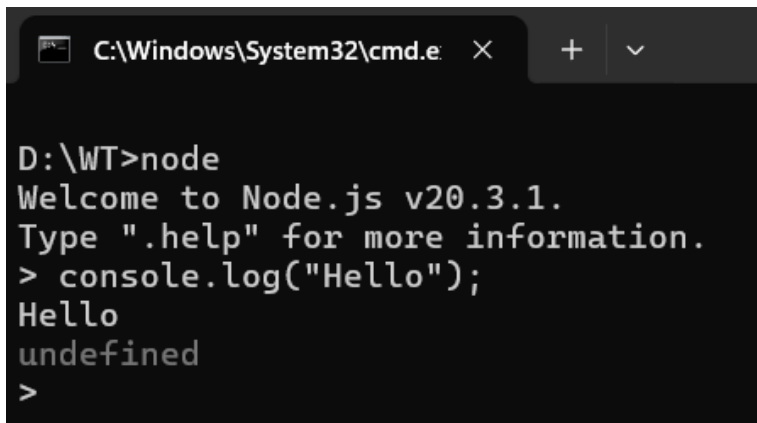
```
let timerId = setTimeout(() => {  
    console.log("This will never execute if cancelled");  
    clearInterval(intervalId);  
}, 5000);
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> node .\async_fun.js  
This will stop after clearInterval() is called  
This will stop after clearInterval() is called  
This will stop after clearInterval() is called  
This will never execute if cancelled  
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> s
```

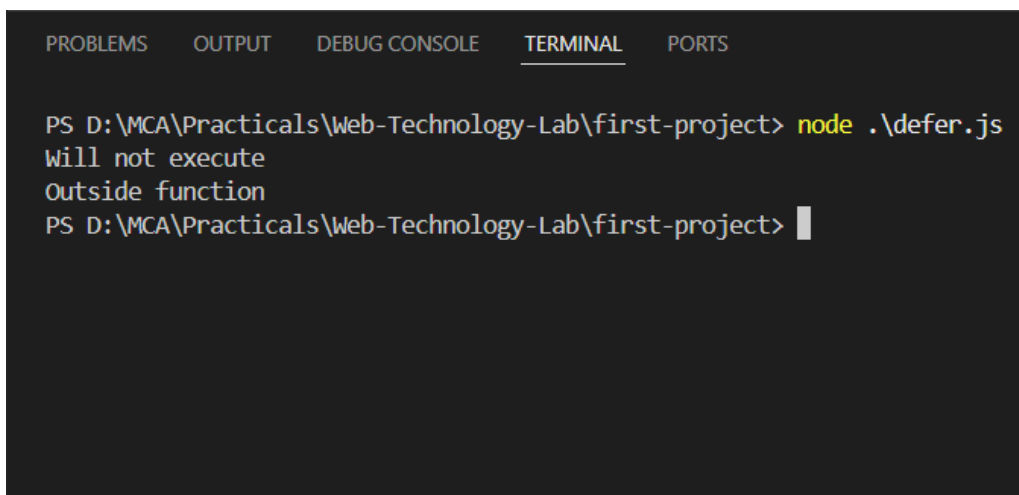
Practical Assignment

Q) Print “Hello” using Node.js REPL

A screenshot of a Windows command prompt window titled 'C:\Windows\System32\cmd.e'. The prompt is 'D:\WT>'. The user has entered 'node', which has started the Node.js REPL. The prompt is now 'Welcome to Node.js v20.3.1. Type ".help" for more information.' The user has entered '> console.log("Hello");', and the output 'Hello' is displayed. The prompt is now 'undefined' followed by '>'.

Q) Write a program to demonstrate Defer Function.

```
function print() {  
    if(!true){  
        console.log("Execute");  
        return;  
    }  
    console.log("Will not execute");  
    return;  
}  
  
print();  
console.log("Outside function");
```

A screenshot of a VS Code terminal window. The terminal has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The prompt is 'PS D:\MCA\Practicals\Web-Technology-Lab\first-project>'. The user has entered 'node .\defer.js'. The output is 'Will not execute' followed by 'Outside function'. The prompt is now 'PS D:\MCA\Practicals\Web-Technology-Lab\first-project>'.

Q) Write a program to find the largest number from the array.

```
var nums = [5, 2, 8, 2, 9, 1]
```

```
var largest = nums[0]
```

```
for (var i = 0; i < nums.length; i++) {
```

```
    if (nums[i] > largest) {
```

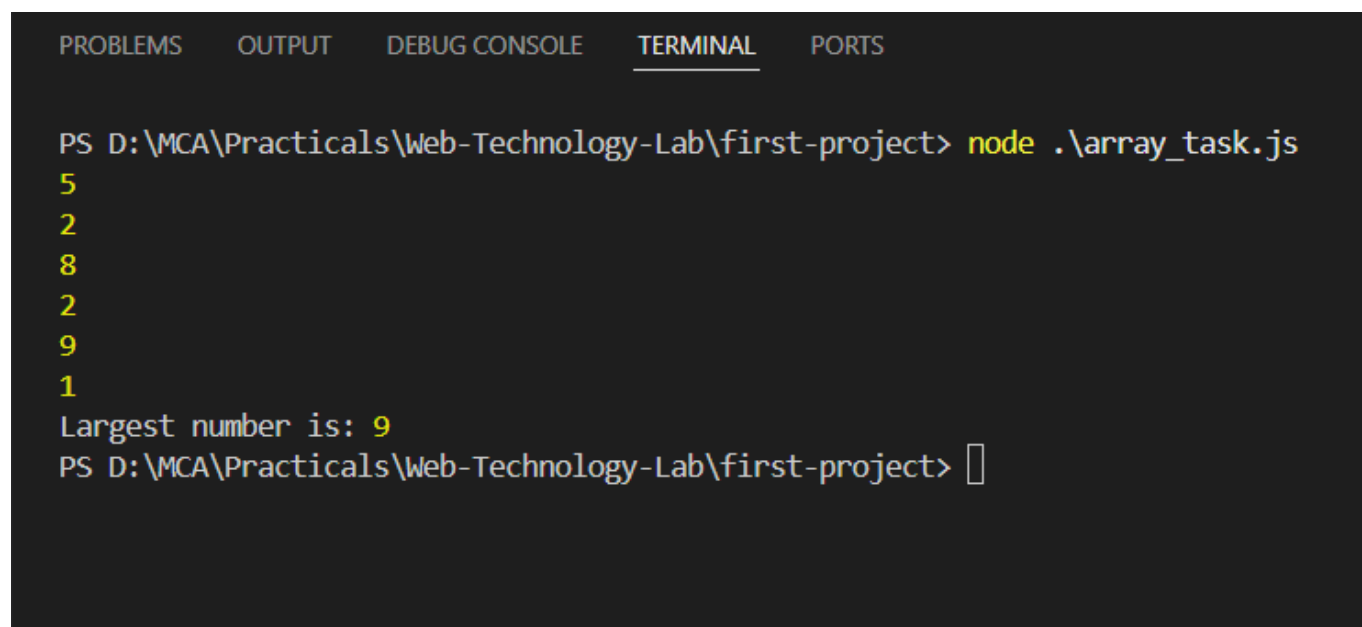
```
        largest = nums[i];
```

```
    }
```

```
    console.log(nums[i]);
```

```
}
```

```
console.log("Largest number is:", largest)
```



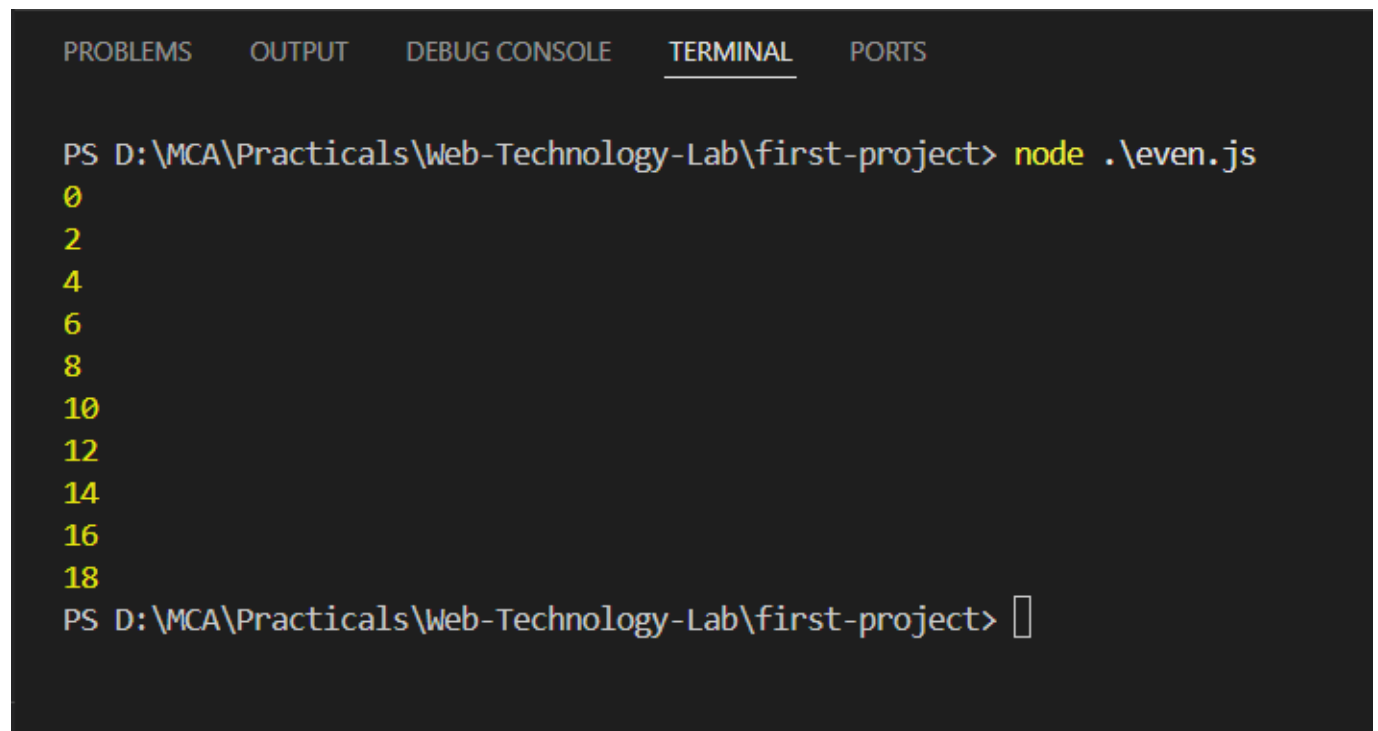
The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The terminal content shows a command prompt 'PS D:\MCA\Practicals\Web-Technology-Lab\first-project>' followed by the command 'node .\array_task.js'. The output of the program is displayed on the next lines: '5', '2', '8', '2', '9', '1', and then 'Largest number is: 9'. The prompt returns to 'PS D:\MCA\Practicals\Web-Technology-Lab\first-project>' with a cursor.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\MCA\Practicals\Web-Technology-Lab\first-project> node .\array_task.js
5
2
8
2
9
1
Largest number is: 9
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> 
```


Q) Write a program to print even number from 0 to 20.

```
for (let i = 0; i < 20; i++) {  
    if(i%2 == 0) {  
        console.log(i);  
    }  
}
```



The screenshot shows a VS Code interface with a terminal window open. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is selected), and PORTS. The command prompt shows the user running 'node .\even.js' in the directory 'D:\MCA\Practicals\Web-Technology-Lab\first-project'. The output of the program is a list of even numbers from 0 to 18, printed on separate lines. The prompt then returns to the command line.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> node .\even.js  
0  
2  
4  
6  
8  
10  
12  
14  
16  
18  
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> 
```

Q) Write a program to find that it's a palindrome or not.

```
function isPalindrome(str) {  
    const reversed = str.split("").reverse().join("");  
    if (reversed == str){  
        return true  
    }  
    return false  
}
```

```
console.log(isPalindrome("racecar"));
```

```
console.log(isPalindrome("level"));
```

```
console.log(isPalindrome("hello"));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> node .\palindrome.js  
true  
true  
false  
PS D:\MCA\Practicals\Web-Technology-Lab\first-project> █
```

Practical 2: File Handling & HTTP Web Server

Q) Write a program in Node.js to Opening a file, reading from a file, writing to a file, closing a file using File Paths, fs Module.

```
const fs = require('fs');
const path = require('path');

const dirName = "myFolder";
const newDirName = "Test";
const fileName = "myFile.txt";
const newFileName = "test.txt";

const filePath = path.join(dirName, fileName);
const renamedFilePath = path.join(newDirName, newFileName);

fs.mkdirSync(dirName);
console.log(`Directory '${dirName}' created successfully.`);

fs.writeFileSync(filePath, 'Helloo from node...');

let message = fs.readFileSync(filePath, 'utf8');
console.log('File Sync :', message);

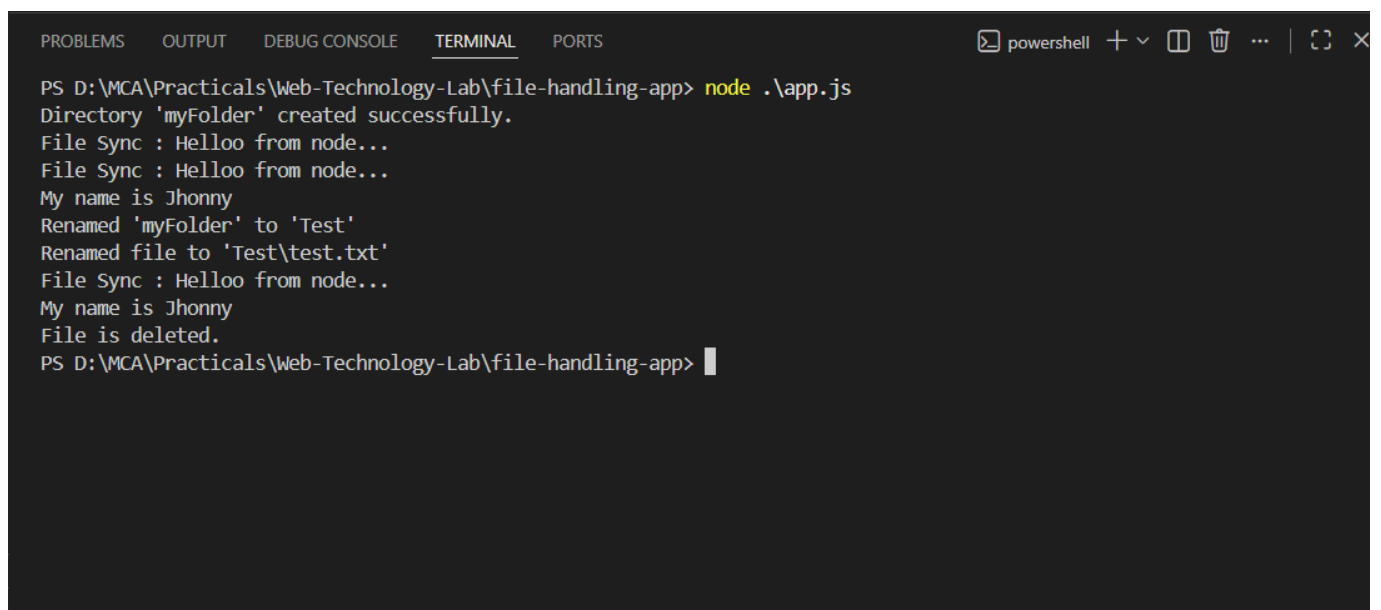
fs.appendFileSync(filePath, '\nMy name is Jhonny');

message = fs.readFileSync(filePath, 'utf8');
console.log('File Sync :', message);

fs.renameSync(dirName, newDirName);
console.log(`Renamed '${dirName}' to '${newDirName}'`);

fs.renameSync(path.join(newDirName, fileName), renamedFilePath);
console.log(`Renamed file to '${renamedFilePath}'`);
```

```
message = fs.readFileSync(renamedFilePath, 'utf8');  
console.log('File Sync :', message);  
  
setTimeout(() => {  
    console.log("File is deleted.");  
    fs.unlinkSync(renamedFilePath);  
    fs.rmdirSync(newDirName);  
}, 10000);
```



The screenshot shows a PowerShell terminal window with the following output:

```
PS D:\MCA\Practicals\Web-Technology-Lab\file-handling-app> node .\app.js  
Directory 'myFolder' created successfully.  
File Sync : Helloo from node...  
File Sync : Helloo from node...  
My name is Jhonny  
Renamed 'myFolder' to 'Test'  
Renamed file to 'Test\test.txt'  
File Sync : Helloo from node...  
My name is Jhonny  
File is deleted.  
PS D:\MCA\Practicals\Web-Technology-Lab\file-handling-app>
```

Q) Write a program in Node.js to HTTP request/response object, Headers, Piping, Shutting down the server.

```
const express = require("express");
const fs = require("fs");
const http = require("http");

const app = express();

app.get("/", (req, res) => {
  console.log("Method:", req.method);
  console.log("URL:", req.url);
  console.log("Headers:", req.headers);

  res.setHeader("Content-Type", "application/json");
  res.setHeader("X-Custom-Header", "Hello-Express");

  res.status(200).json({
    message: "Hello from Express",
    userAgent: req.headers["user-agent"]
  });
});

app.get("/stream", (req, res) => {
  res.setHeader("Content-Type", "text/plain");

  const readStream = fs.createReadStream(__filename);
  readStream.pipe(res);

  readStream.on("error", (err) => {
    res.status(500).end("Stream error");
  });
});
```

```
app.get("/raw", (req, res) => {
  res.write("This is written using res.write()\n");
  res.write(`Time: ${new Date().toISOString()}\n`);
  res.end("Done\n");
});

const server = http.createServer(app);

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

function shutdown(signal) {
  console.log(`\nReceived ${signal}. Shutting down gracefully...`);

  server.close(() => {
    console.log("HTTP server closed.");
    process.exit(0);
  });

  setTimeout(() => {
    console.error("Force shutdown");
    process.exit(1);
  }, 10000);
}

process.on("SIGINT", shutdown); // Ctrl+C
process.on("SIGTERM", shutdown);
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\MCA\Practicals\Web-Technology-Lab\database-app> node .\server.js
Server running on http://localhost:3000
Method: GET
URL: /
Headers: {
  host: 'localhost:3000',
  connection: 'keep-alive',
  'sec-ch-ua': '"Brave";v="143", "Chromium";v="143", "Not A(Brand";v="24"',
  'sec-ch-ua-mobile': '?0',
  'sec-ch-ua-platform': 'Windows',
  'upgrade-insecure-requests': '1',
  'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8',
  'sec-gpc': '1',
  'accept-language': 'en-US,en;q=0.5',
  'sec-fetch-site': 'none',
  'sec-fetch-mode': 'navigate',
  'sec-fetch-user': '?1',
  'sec-fetch-dest': 'document',
  'accept-encoding': 'gzip, deflate, br, zstd'
}
```

```
< > ↺ localhost:3000
Pretty-print ✓
{
  "message": "Hello from Express",
  "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
}
```

```
< > ↻ localhost:3000/stream

const express = require("express");
const fs = require("fs");
const http = require("http");

const app = express();

app.get("/", (req, res) => {
  console.log("Method:", req.method);
  console.log("URL:", req.url);
  console.log("Headers:", req.headers);

  res.setHeader("Content-Type", "application/json");
  res.setHeader("X-Custom-Header", "Hello-Express");

  res.status(200).json({
    message: "Hello from Express",
    userAgent: req.headers["user-agent"]
  });
});

app.get("/stream", (req, res) => {
  res.setHeader("Content-Type", "text/plain");

  const readStream = fs.createReadStream(__filename);
  readStream.pipe(res);

  readStream.on("error", (err) => {
    res.status(500).end("Stream error");
  });
});

app.get("/raw", (req, res) => {
  res.write("This is written using res.write()\n");
  res.write(`Time: ${new Date().toISOString()}\n`);
  res.end("Done\n");
});

const server = http.createServer(app);

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

```
< > ↻ localhost:3000/raw

This is written using res.write()
Time: 2025-12-21T07:37:54.231Z
Done
```



```
'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8',
  'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8',
  'sec-gpc': '1',
  'accept-language': 'en-US,en;q=0.5',
  'sec-fetch-site': 'none',
  'sec-fetch-mode': 'navigate',
  'sec-fetch-user': '?1',
  'sec-fetch-dest': 'document',
  'accept-encoding': 'gzip, deflate, br, zstd'
}
```

Received SIGINT. Shutting down gracefully...

HTTP server closed.

PS D:\MCA\Practicals\Web-Technology-Lab\database-app> █

Practical 3: Database.

Q) Write a program in Node.js to connect MySQL database

```
const mysql = require('mysql2');  
  
const con = mysql.createConnection({  
  host: 'localhost',  
  user: 'root',  
  password: 'admin',  
  database: 'test'  
});  
  
con.connect(function(err) {  
  if (err) throw err;  
  console.log("Connected to MySQL using mysql2");  
});
```

```
PS D:\WT> npm install mysql2  
>>  
  
added 12 packages in 3s  
  
2 packages are looking for funding  
run `npm fund` for details
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  
  
PS D:\WT> node DB.js  
Connected to MySQL using mysql2  
█
```

Q) Write a program in Node.js to connect Mysql database and insert record

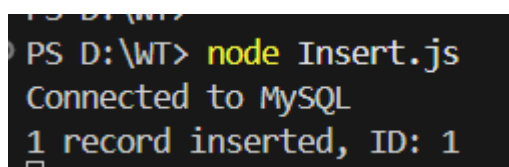
```
const mysql = require('mysql2');

const con = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'admin',
  database: 'test1'
});

con.connect(function(err) {
  if (err) throw err;
  console.log("Connected to MySQL");

  // Insert single record
  const sql = "INSERT INTO students (name, marks) VALUES (?, ?)";
  const values = ['Alice', 88];

  con.query(sql, values, function(err, result) {
    if (err) throw err;
    console.log("1 record inserted, ID: " + result.insertId);
  });
});
```



A terminal window with a dark background. The prompt is 'PS D:\WT>'. The command 'node Insert.js' has been executed. The output shows 'Connected to MySQL' on the first line and '1 record inserted, ID: 1' on the second line. A cursor is visible at the end of the second line.

```
PS D:\WT> node Insert.js
Connected to MySQL
1 record inserted, ID: 1
```

6

7 • `select * from students ;`

Result Grid



Filter Rows:

	id	name	marks
▶	1	Alice	88
•	NULL	NULL	NULL

Q) Write a program in Node.js to connect Mysql database and insert multiple record

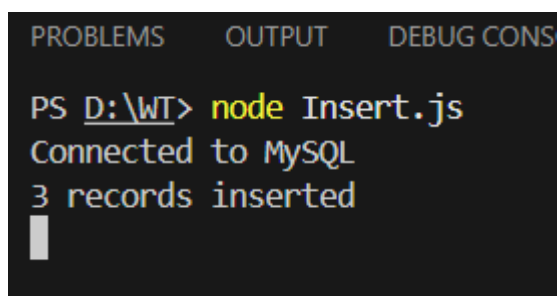
```
const mysql = require('mysql2');

const con = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'admin',
  database: 'test1'
});

// Connect to database
con.connect(function(err) {
  if (err) throw err;
  console.log("Connected to MySQL");

  const sql = "INSERT INTO students (name, marks) VALUES ?";
  const values = [
    ['Ravi', 78],
    ['Neha', 92],
    ['Sita', 85]
  ];

  con.query(sql, [values], function(err, result) {
    if (err) throw err;
    console.log(result.affectedRows + " records inserted");
  });
});
```



```
PROBLEMS  OUTPUT  DEBUG  CONS
PS D:\WT> node Insert.js
Connected to MySQL
3 records inserted
```

7 • `select * from students ;`

Result Grid



Filter Rows:

	id	name	marks
▶	1	Alice	88
	2	Alice	88
	3	Ravi	78
	4	Neha	92
	5	Sita	85
*	NULL	NULL	NULL

Q) Write a program in Node.js to connect Mysql database and perform crud operation

```
const mysql = require('mysql2');

const con = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'admin',
  database: 'test1'
});

con.connect(function(err) {
  if (err) throw err;
  console.log("Connected to MySQL");

  // ===== CREATE =====

  const insertSql = "INSERT INTO students (name, marks) VALUES (?, ?)";
  con.query(insertSql, ['Alice', 88], function(err, result) {
    if (err) throw err;
    console.log("Record inserted, ID: " + result.insertId);

    // ===== READ =====

    con.query("SELECT * FROM students", function(err, rows) {
      if (err) throw err;
      console.log("All Students:");
      console.log(rows);

      // ===== UPDATE =====

      const updateSql = "UPDATE students SET marks = ? WHERE name = ?";
      con.query(updateSql, [95, 'Alice'], function(err, result) {
        if (err) throw err;
        console.log("Record updated");

        // READ AFTER UPDATE

        con.query("SELECT * FROM students", function(err, rows) {
```

```

    if (err) throw err;

    console.log("All Students After Update:");

    console.log(rows);

    // ===== DELETE =====

    const deleteSql = "DELETE FROM students WHERE name = ?";

    con.query(deleteSql, ['Alice'], function(err, result) {

        if (err) throw err;

        console.log("Record deleted");

        // READ AFTER DELETE

        con.query("SELECT * FROM students", function(err, rows) {

            if (err) throw err;

            console.log("All Students After Delete:");

            console.log(rows);

        });

    });

});

});

});

});

});

});

});

```

OUTPUT :

```

PS D:\WT> node crud.js
Connected to MySQL
Record inserted, ID: 1
All Students:
[ { id: 1, name: 'Alice', marks: 88 } ]
Record updated
All Students After Update:
[ { id: 1, name: 'Alice', marks: 95 } ]
Record deleted
All Students After Delete:
[]

```


Practical 4: Introduction to ReactJs

Q) Write a program in React.js to demonstrate “Hello World” using JSX.

App.js

```
import logo from './logo.svg';
```

```
import './App.css';
```

```
function App() {
```

```
  return (
```

```
    <div className="App">
```

```
      <header className="App-header">
```

```
        <img src={logo} className="App-logo" alt="logo" />
```

```
        <p>
```

```
          Edit <code>src/App.js</code> and save to reload.
```

```
        </p>
```

```
        <a
```

```
          className="App-link"
```

```
          href="https://reactjs.org"
```

```
          target="_blank"
```

```
          rel="noopener noreferrer"
```

```
        >
```

```
          Learn React
```

```
        </a>
```

```
      </header>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

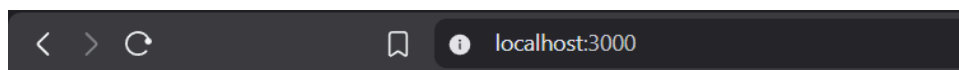
main.jsx

```
const MyElement = () => {  
  return <h1>Hello, world!, This is main.jsx</h1>;  
}
```

```
export default MyElement;
```

index.js

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import './index.css';  
import MyElement from './main';  
import reportWebVitals from './reportWebVitals';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode>  
    <MyElement />  
  </React.StrictMode>  
);  
  
reportWebVitals();
```



Hello, world!, This is main.jsx

Practical 5:

Q) Create a HTML file, in which onclick and alert has been used.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Document</title>

</head>

<body>

  <ul>

    <li>

      Minecraft

      <br/>

      <br/>

      <button onclick="minecraftButton()">Minecraft</button>

    </li>

    <br/>

    <li>

      Football

      <br/>

      <br/>

      <button onclick="footballButton()">Football</button>

    </li>

    <br/>

    <li>

      Car

      <br/>
```

```

```

```
<br/>
```

```
<button onclick="carButton()">Car</button>
```

```
</li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

```
<script>
```

```
document.getElementById("minecarft").onclick=function() {  
    alert("HEIN!!")  
}
```

```
document.getElementById("football").onclick=function() {  
    alert("HALA! MADRID!!!")  
}
```

```
document.getElementById("car").onclick=function() {  
    alert("'BRUUMM BRUMMMM BRUUMMM!!!")  
}
```

```
function minecraftButton(){  
    alert("HEIN!!")  
}
```

```
function footballButton(){  
    alert("HALA! MADRID!!!")  
}
```

```
function carButton(){  
    alert("'BRUUMM BRUMMMM BRUUMMM!!!")  
}
```

```
</script>
```

FileD:/MCA/Practicals/Web-Technology-Lab/event-app/index.html

• Minecraft



Minecraft

• Football



This page says

HEIN!!

OK

Page 29 of 37

Q) Create a react app and use the event like onclick and Alert.

```
function App() {  
  const glowStyle = {  
    boxShadow: "0 0 20px 6px rgba(0, 255, 255, 0.8)",  
    transform: "scale(1.03)",  
    transition: "all 0.3s ease",  
    cursor: "pointer",  
    borderRadius: "8px",  
  };  
  
  const normalStyle = {  
    transition: "all 0.3s ease",  
    cursor: "pointer",  
    borderRadius: "8px",  
  };  
  
  const minecraftAlert = () => alert("HEIN!!!");  
  const footballAlert = () => alert("HALA! MADRID!!!");  
  const carAlert = () => alert("BRUUMM BRUMMMM BRUUMMM!!!");  
  
  const handleEnter = (e) => {  
    Object.assign(e.target.style, glowStyle);  
  };  
  
  const handleLeave = (e) => {  
    Object.assign(e.target.style, normalStyle);  
  };  
  
  return (  
    <div>  
      <ul>
```


Minecraft

<button onClick={minecraftAlert}>Minecraft</button>

Football

 <button onClick={footballAlert}>Football</button>

 Car

 <button onClick={carAlert}>Car</button>

</div>

);
}

export default App;

```



localhost:3000

• Football



Football

• Car



localhost:3000 says  
HALA! MADRID!!!  
OK

localhost:3000



Football

• Car



Car

### **Practical Assignment:**

#### **Q) Create a Simple Calculator using React.js**

```
import { useState } from "react";

function App() {
 const [num1, setNum1] = useState("");
 const [num2, setNum2] = useState("");
 const [operation, setOperation] = useState("add");
 const [result, setResult] = useState(null);

 const calculate = () => {
 const a = Number(num1);
 const b = Number(num2);

 let res = 0;

 switch (operation) {
 case "add":
 res = a + b;
 break;
 case "sub":
 res = a - b;
 break;
 case "mul":
 res = a * b;
 break;
 case "div":
 res = b !== 0 ? a / b : "Cannot divide by 0";
 break;
 default:
 res = 0;
 }
 }
}
```

```

 }

 setResult(res);
 };

 return (
 <div>
 <h2>React Calculator</h2>

 <input
 type="number"
 placeholder="First number"
 value={num1}
 onChange={(e) => setNum1(e.target.value)}
 />

 <input
 type="number"
 placeholder="Second number"
 value={num2}
 onChange={(e) => setNum2(e.target.value)}
 />

 <select value={operation} onChange={(e) => setOperation(e.target.value)}>
 <option value="add">Add (+)</option>
 <option value="sub">Subtract (-)</option>
 <option value="mul">Multiply (×)</option>
 <option value="div">Divide (÷)</option>
 </select>

 <button onClick={calculate}>Calculate</button>
 </div>
);
}

```

```
 {result !== null && <h3>Result: {result}</h3>}
 </div>
);
}
```

```
export default App;
```



## React Calculator

Add (+) ▾

Calculate

**Result: 90**



## React Calculator

Multiply (×) ▾

Calculate

**Result: 2000**

## React Calculator

Subtract (-) ▾

Calculate

**Result: -10**

## React Calculator

Divide (÷) ▾

Calculate

**Result: 0.8**